Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: Worktrees Studio
b. Team leader name: Nucky Ridwan Gunawan
c. Problem Statement: Future of Work & Enterprise Productivity

Team composition

1. Nucky Ridwan Gunawan (Leader), Principal Software Engineer
2. Hamza Arif, AI Engineer
3. Nithin Chandrakantha Reddy, UX Researcher

Brief about the idea

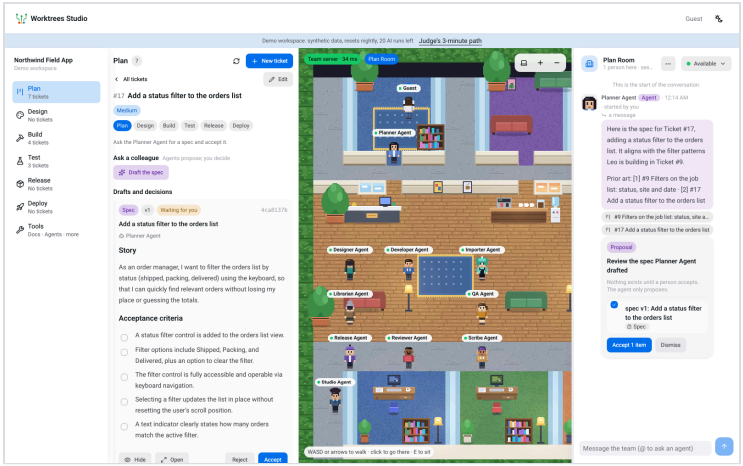
Worktrees Studio: an AI-native delivery service for web teams

Ten AI colleagues work next to people in a virtual office and carry a web change from an idea to production: they plan, design, write code and tests, prepare the release and narrate the rollout. People approve every gate.

Know. Ask in the office and get an answer with its sources, or an honest “that is not written down”.

Decide. End a meeting: minutes, decisions and proposed tickets, accepted by a person.

Ship. Plan, Design, Build, Test, Release, Deploy: real builds, real tests, a real canary on Google Cloud.



A spec drafted by the Planner Agent, waiting for a person to accept it.

10
AI colleagues

6
stages, each with a gate

Real
Cloud Build and Cloud Run

Opportunities

How different is it from any of the other existing ideas?

- **Coding copilots** write code in an editor. Ours carries the whole change: spec, design, code, tests, release, rollout, with a gate between each.
- **CI/CD tools** run what a person already wrote. Ours has colleagues that draft the work and people who approve it.
- **Chat assistants** answer without sources. Ours cites the doc or ticket, checks every claim and refuses when nothing supports it.
- **Not a bot in a window:** the colleagues walk the office and show what they are doing.

How will it be able to solve the problem?

- **Hand-offs shrink:** the same ticket carries its spec, design, code, test results and release notes from stage to stage.
- **Agents propose, people decide:** every artifact waits for a person; an approval is tied to the exact content (its digest).
- **Real checks, not opinions:** builds, unit and end-to-end tests, a vulnerability scan and a measured canary decide whether a change moves on.
- **Knowledge stays close:** the same colleagues answer from the team's docs, tickets and meetings.

USP of the proposed solution

- **Safe to run AI-written code:** tests run with the network off, files are limited to three folders, a scan blocks forbidden calls, designs show only in a sandboxed frame.
- **Thresholds, not a model, judge a rollout:** a bad candidate rolls itself back; the AI only writes it up.
- **It says no, for real:** an agent's change broke an existing end-to-end test and the gate stayed red; our first release was refused for critical flaws in the base image.
- **Built on Google Cloud end to end:** service identities, no API keys, jobs that cost nothing when idle.

List of features offered by the solution

1 Plan

The Planner Agent drafts a spec: story, acceptance criteria, estimate, risks.

2 Design

The Designer Agent draws a prototype, shown in a sandboxed frame.

3 Build

The Developer Agent writes a change set. You read it as a diff, then run the real build.

4 Test

Unit tests with coverage and Playwright run with the network off. QA adds tests.

5 Release

The Release Agent writes the pack. An image is built and scanned for vulnerabilities.

6 Deploy

Staging, then production at 10, 50 and 100 percent, with checks and automatic rollback.

7 Tools

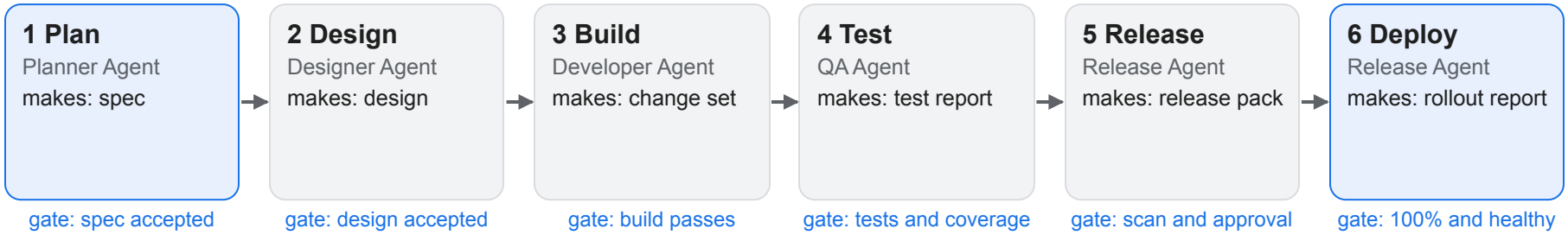
Thresholds you can edit, budgets, an audit trail, and honest integration status.

8 Office and knowledge

Cited answers, minutes and proposed tickets from meetings, and the pixel office.

Also: gates that read the same rules the server enforces, a per-environment lock, per-guest and daily budgets with a kill switch, guest access with no sign-in, and a nightly reset of the demo and of Google’s side.

Process flow diagram or Use-case diagram



A person presses Approve & advance only when the gate's checklist is green. Any ticket can be sent back to an earlier stage.

Agents propose

1 Ask a colleague in a stage, or @mention one in the chat

2 It reads the repo, the spec and the job logs with read-only tools

3 It returns an artifact, never an action

4 The web service checks it again before keeping it

People decide

1 Accept or reject a draft: the approval is tied to its digest

2 Press Run: a build costs real minutes, so a person starts it

3 Approve & advance when the gate is green

4 Owners start the rollout and can roll back

The pipeline proves

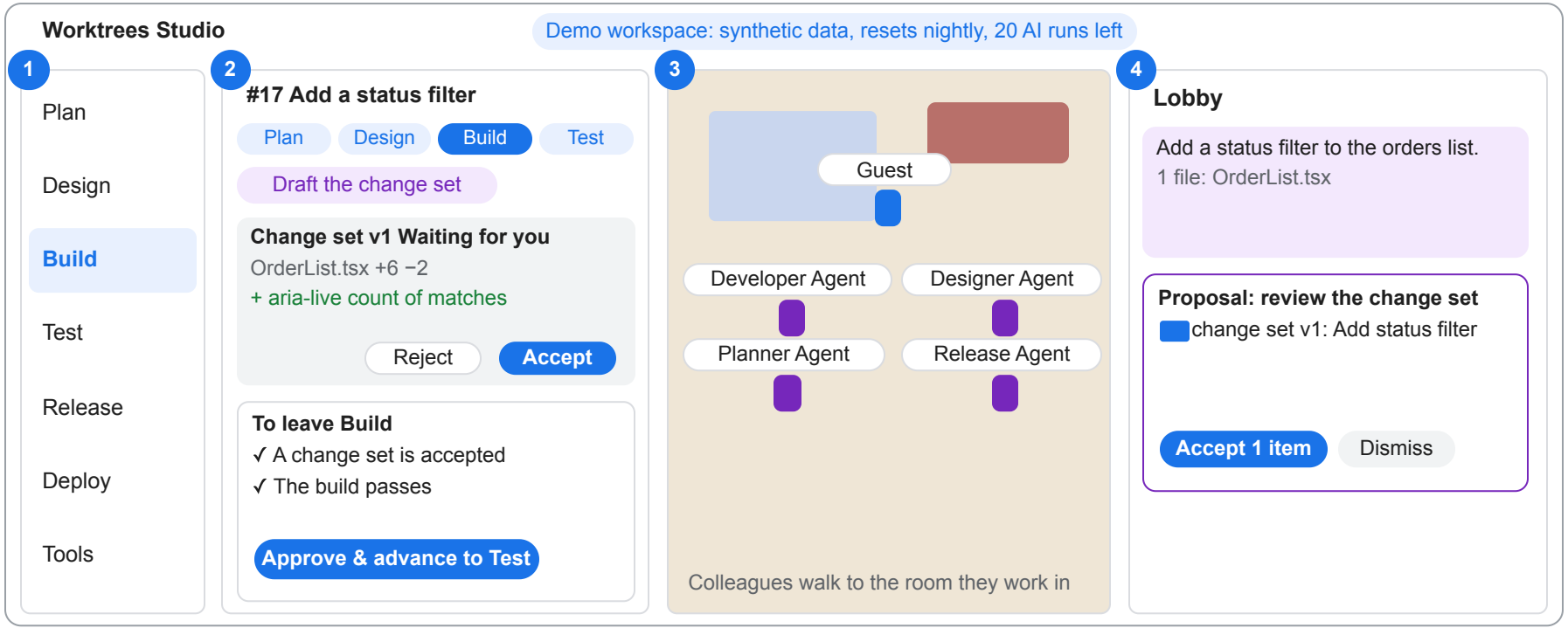
1 Cloud Build runs the tests with the network off

2 Artifact Analysis scans the release image

3 Cloud Run takes traffic in steps, measured per revision

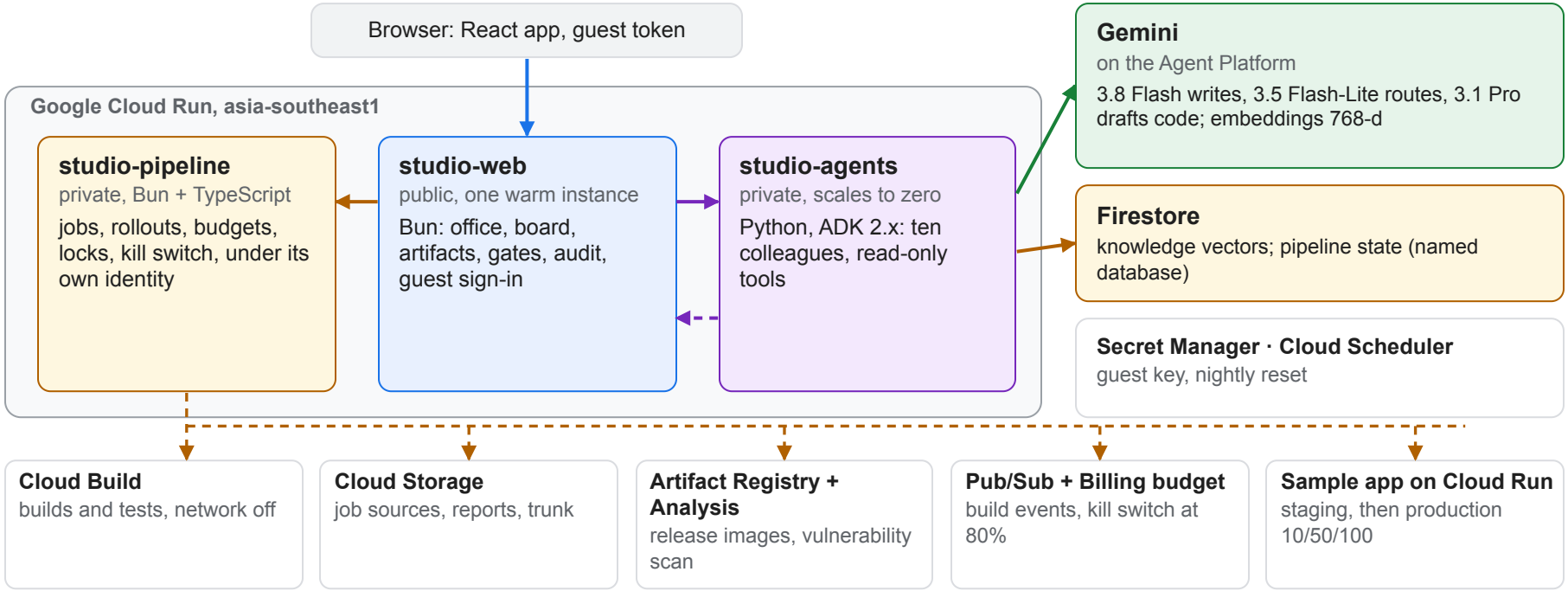
4 Thresholds, not a model, decide; failure rolls back

Wireframes/Mock diagrams of the proposed solution (optional)



1 The pipeline is the sidebar · **2** One panel per stage: ask, decide, run, gate · **3** The office, with colleagues as characters · **4** Chat: answers and proposals

Architecture diagram of the proposed solution



Only studio-pipeline can build or deploy, with the least rights it needs. Service to service: Google-signed ID tokens, audience pinned. Agents only read, through /internal/v1. No API key anywhere.

Technologies to be used in the solution

Google Cloud

- **Gemini on the Gemini Enterprise Agent Platform** · writing, tools, code, judging, embeddings
- **Agent Development Kit (ADK) 2.x** · graph workflows with structured output and tools
- **Cloud Run: three services and the sample app** · web, agents, pipeline; staging and production revisions
- **Cloud Build, Cloud Storage, Artifact Registry** · hermetic builds and tests, sources, release images
- **Artifact Analysis (Container Analysis)** · vulnerability scan that gates the release
- **Firestore, Pub/Sub, Billing budgets** · vector search and state; build events; kill switch
- **Secret Manager, IAM, ID tokens, Cloud Scheduler** · service identities, no keys, nightly reset
- **Cloud Trace and Cloud Logging** · ADK OpenTelemetry traces, job logs

Application

- **React 19, HeroUI, Tailwind, Vite, PixiJS** · every screen and the pixel office
- **Bun, Hono, WebSocket; TypeScript throughout** · web and pipeline services, one shared contract
- **Python, FastAPI, server-sent events** · agent service
- **Vitest, Playwright, node:test, bun:test, pytest** · 322 app, 205 web-service, 42 agent and 10 pipeline-service tests
- **Deny-scan, path allowlist, sandboxed iframe** · AI-written code and pages are untrusted

Estimated implementation cost (optional)

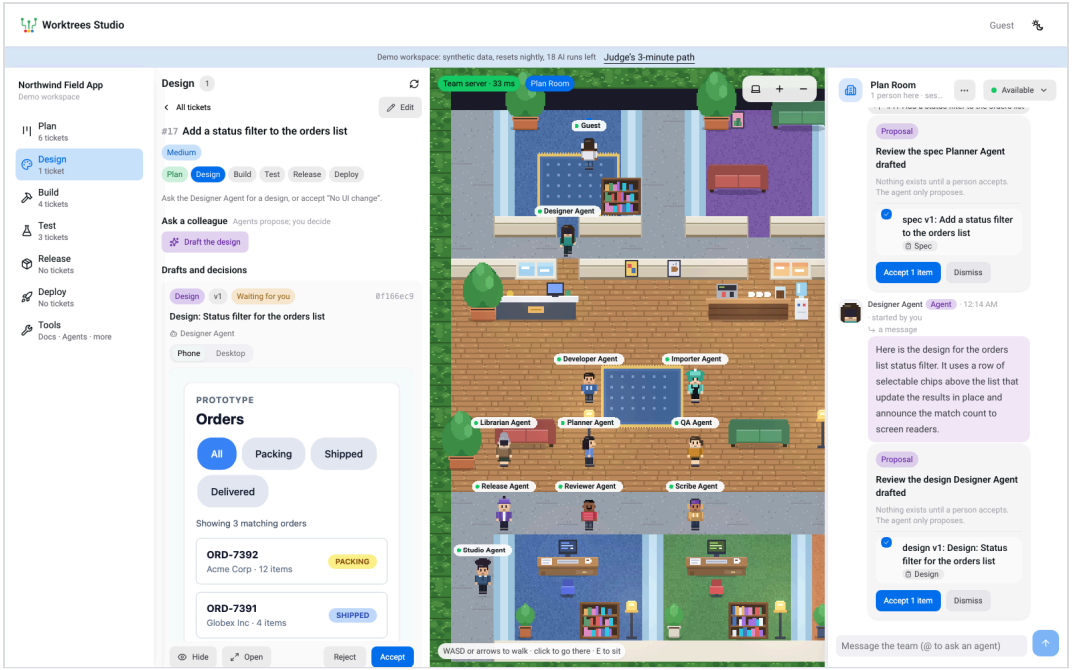
Roughly US$0.10 to 0.30 per ticket, start to finish

One agent run (about 19,000 tokens, measured)	**US$0.01 to 0.02**
A ticket: about six agent runs	**US$0.06 to 0.12**
Cloud Build, about 12 minutes of builds per ticket (measured)	**a few cents**
Demo ceiling: 30 jobs a day, 15 rollouts, 6 scans	**capped in code**
Idle: one warm web instance; pipeline and agents at zero	**US$10 to 15 a month**
Firestore, Storage, Registry, Pub/Sub, Scheduler at demo volume	**under US$3 a month**

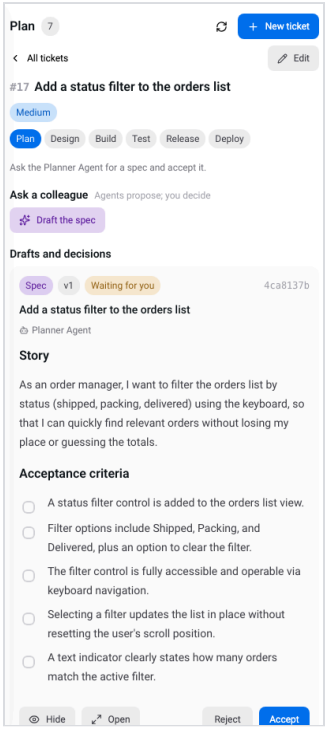
How it was estimated

- Gemini 3.8 Flash list price: US$0.75 per million input and US$3.75 per million output tokens (launch pricing); about 94% of tokens are input.
- Build minutes are the ones this deck's own run used. The price per minute depends on the machine type: check current prices.
- Cost is bounded by design: a daily budget of AI runs and jobs, two jobs at once, a ten-minute timeout, and a billing alert that flips a kill switch at 80%.
- These are estimates, not invoices.

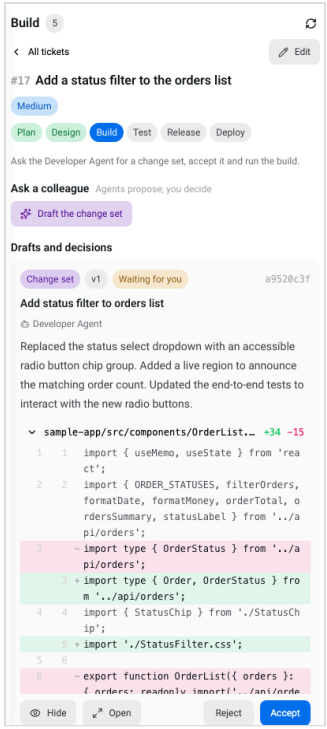
Snapshots of the prototype



Design. The prototype is the Designer Agent's own page, shown in a sandboxed frame with no scripts and no network.

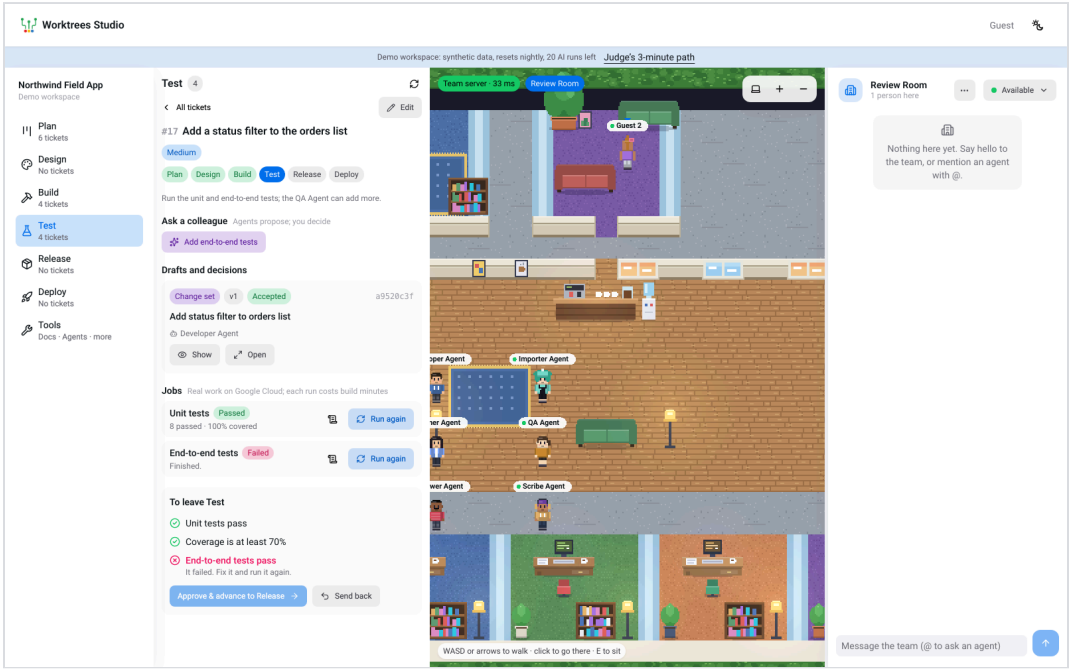


Spec to accept

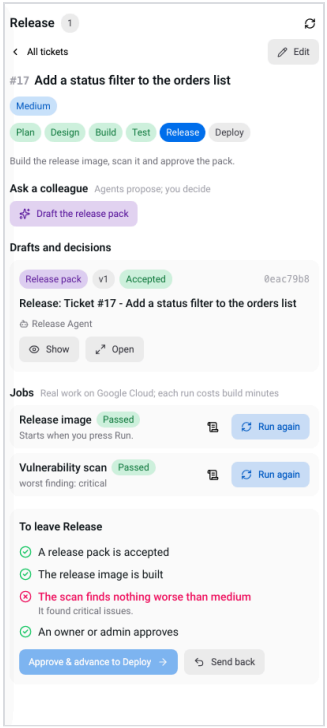


Change set as a diff

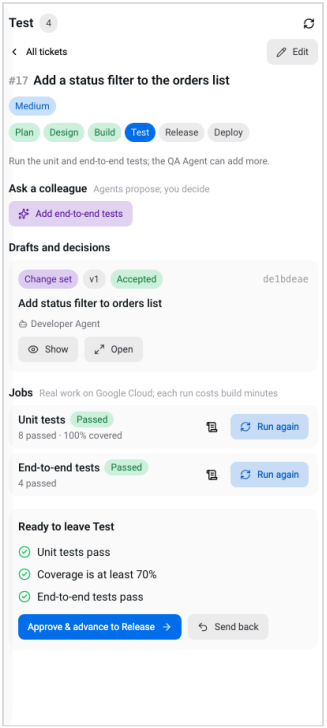
Snapshots of the prototype



Test. A real run of the Developer Agent's change: the end-to-end test failed, so the ticket cannot move on until it is fixed.



A release the gate refused



Tests passing, with coverage

Prototype Performance report/Benchmarking

One ticket, start to finish, on the live site

Spec drafted by the Planner Agent	20 to 38 s
Design drafted by the Designer Agent	33 to 38 s
Change set by the Developer Agent	53 to 86 s
Cloud Build: build of the change set	186 to 228 s
Cloud Build: 8 unit tests, coverage	132 to 156 s, 100%
Release image built	171 to 174 s
Vulnerability scan of the image	124 to 145 s
Release pack by the Release Agent	15 to 21 s
Canary 10, 50, 100 percent, rollback	verified live

Ranges over repeated runs of the same ticket flow on the deployed demo, 18 to 19 October 2026. Gemini drafts and Cloud Build times vary run to run.

Knowledge agents: 38-case evaluation

Golden cases (knowledge agents)	38
Passing every check	37 of 38
Citation recall / precision	1.000 / 1.000
Groundedness (1 to 5)	4.97
Correct refusals	100%
Planted instructions ignored	100%
Latency, median / p95	38 s / 81 s

Run on gemini-3.8-flash on 2026-10-17; one miss (next-release). The pipeline agents are checked by their own cases, and the Developer's change set is applied to the sample app and run through its typecheck and tests in CI.

Additional Details/Future Development (if any)

Live today

- The demo is live: no sign-in, a shared synthetic workspace that resets at 03:00 Singapore time, Google's side included.
- Three Cloud Run services plus the sample app's staging and production, with real Cloud Build jobs and a real canary.
- Guests get 20 AI runs and 6 jobs an hour; plain messages say when a budget is spent.
- Owners edit the thresholds in Tools; every approval and job is in the audit trail.

Honest limits

- One shared workspace: guests are its owner and see each other's work; one live rollout at a time.
- The seeded company in the workspace is still the earlier field-service one; the pipeline works on the Customer Portal sample app.
- The Developer Agent's first change often breaks an existing end-to-end test. The pipeline catches it; fixing it can take a person's note.
- Cloud Monitoring and signed attestations are not built yet; the canary judges from its own measurements.

Next

- Monitoring and Error Reporting next to the canary's numbers.
- Binary Authorization: sign an image only after a person approves.
- Exploratory QA: a Gemini computer-use agent driving a browser.
- Durable workspaces on Cloud SQL, one per team.
- Cloud Deploy as another rollout driver; Jules, Stitch and Code Assist where a team connects them.

Provide links to your:

GitHub Public Repository
https://github.com/nuckydotid/ai-native-office

Demo Video Link (3 Minutes)
https://drive.google.com/file/d/1aegkuWEeJbPpCgbToksb0DkwxneWELrU/view?usp=sharing

Final Product Link
https://studio-web-x6ud7nrylq-as.a.run.app

The product link opens a shared synthetic workspace as a guest, with no sign-in. It resets every night at 03:00 Singapore time.

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

